

[← Go Back](#)

Hardware

Tutorials

[Debugging SAM-Based Arduino® Boards with Atmel-ICE](#)[Accessing the Built-in RGB LED on the MKR WiFi 1010](#)[How to Connect Sensors to the MKR WiFi 1010](#)[Connecting MKR WiFi 1010 to a Wi-Fi Network](#)[MKR WiFi 1010 Bluetooth® Low Energy](#)[Host a Web Server on the MKR WiFi 1010](#)[MKR WiFi 1010 Battery Application Note](#)[Using the Segger J-Link Debugger with the MKR Boards](#)[Sending Data over MQTT](#)[Serial to OLED with MKR WiFi 1010](#)[Powering MKR WiFi 1010 with Batteries](#)[RTC \(Real Time Clock\) with MKR WiFi 1010 and OLED Display](#)[Scanning Networks with MKR WiFi 1010](#)[Securely Connecting an Arduino MKR WiFi 1010 to AWS IoT Core](#)[Web Server Access Point \(AP\) Mode with MKR WiFi 1010](#)[Home](#) / [Hardware](#) / [MKR WiFi 1010](#) /

Powering MKR WiFi 1010 with Batteries

Powering MKR WiFi 1010 with Batteries

Learn how to power the MKR WiFi 1010 with batteries, and how to use the low-power features of the board.

Author · Simone Majocchi · Last revision · 12/19/2023

ON THIS PAGE

Introduction

[Hardware & Software Needed](#)[Circuit](#)[Schematic](#)[Let's Start!](#)[Conclusion](#)

Introduction

The MKR WiFi 1010 board is designed to run on a Li-Po battery.

Your MKR board has all the circuitry to use a Li-Po battery, charging it when there is power coming from VIN or USB, or using it as main power supply when there is no other source available. If you are going to use a Li-Po battery, it is useful to understand how to reduce power consumption using the `ArduinoLowPower` Library and the `Wi-Fi LowPowerMode`. Some simple practices will make your project battery friendly.

Hardware & Software Needed

Arduino MKR WiFi 1010

Li-Po battery 1024 mAh

[Help](#)

Power source with micro-USB cable

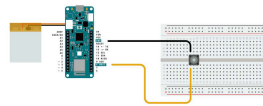
Arduino IDE (offline and online versions available)

Arduino SAMD Board

Package installed, [follow this link for instructions](#)

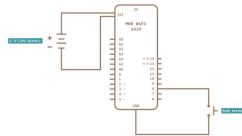
WiFiNINA library (explained later in this tutorial)

Circuit



Circuit with board, Li-Po battery and a button.

Schematic



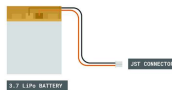
Schematic of our circuit.

Let's Start!

This tutorial is more of a conceptual one than a practical one, because there is not much we are going to do on the hardware side: just connect your Li-Po battery to the connector and see the board spring up to life. This happens if the battery is charged at some level and it is unlikely that your battery is so discharged that

one of the sketches goes between D8 and GND.

The first step is to connect the board to a source of energy, this can be your computer or a phone charger. In both cases there will be a cable that ends with a micro USB that goes into the MKR WiFi 1010 socket. A standard 500 mA supply is ok and will charge your battery in a number of hours that is proportional to the capacity of the battery. The circuit is designed to provide 4.2V and 512 mAh; with a typical C/2 charge/discharge rating of the cells, this is the reason why we suggest a 1024 mAh minimum capacity. The hours taken for the charge are therefore the capacity divided by the charging current. You can find batteries with different C values depending on their chemistry and application, but the ones available for powering microcontroller boards are usually rated at C/2. Operating the Li-Po battery within its specs will grant it a long and cool (no heating) life.



LiPo battery with JST PH connector.

When the battery is charging, you will see the charging LED lit. When no charge is going on, either because the battery is full or the charging timeout has been reached, that LED is off while the

At this point our MKR WiFi 1010 is still connected to a power supply through the micro-USB port and the Li-Po cell is just connected but not used.

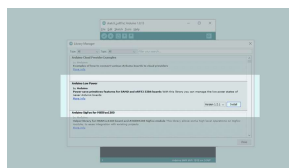
As soon as we disconnect the USB power, the battery power kicks in and we have an uninterrupted supply to all the components on the board. This is important to understand: no reset is needed when going from one power source to another, so you can be creative in your charging solutions, like solar cells, wind turbines, bicycle dynamos and so on. Just remember that the VIN and USB expect a clean and constant 5V (no more, no less).

We can now go to the software part and start checking the availability in our programming environment, of the two libraries we need:

ArduinoLowPower

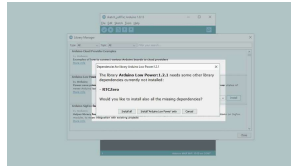
WiFinINA

Go to **Sketch > Include Library > Manage Libraries...** and the Library manager will open. Type in the search box the name of each library, as written above, and check if they are already installed (you see the "INSTALLED" label near the name), or alternatively just install the latest version.



Install the Arduino Low

When installing the Arduino Low Power Lib, it will automatically check if you have the RTCZero Lib, which is a dependency. This is a great feature of the latest lib manager.



Install the RTCZero library.

Now everything is ready to begin the testing.

The NINA module can just turn off the WiFi, but has no deep sleep mode, while the SAMD microcontroller has one. This means that we have different levels of power saving.

Use `WiFi.lowPowerMode()`

This is an automatically managed mode where the Wi-Fi NINA Module reduces its power drain, bringing the overall power consumption to 30mA. Any incoming data is received and the device regularly sends out to the beacon signal, each 100ms, to keep the AP connection alive.

Use `WiFi.end()`

If `WiFi.begin()` was used to connect to an access point, the connection will be disconnected. If `WiFi.beginAP()` was used before to create an access point, the `WiFi.end()` will stop listening it too. In both cases we need to reinitialize

Use `LowPower.sleep()`

Puts the MCU in sleep mode. The sleep mode allows power optimization with a slower wakeup time. Only the chosen peripherals are on.

Use `LowPower.deepSleep()`

Puts the MCU in deep sleep mode. The deep sleep mode allows power optimization with the slowest wake-up time. All but the RTC peripherals are stopped. The CPU can wakeup only using RTC or wakeup on interrupt capable pins.

You find full reference for these APIs in the libraries documentation: here for [WiFiNINA](#) and here for [ArduinoLowPower](#).

These APIs can be combined, one for each type, to achieve the power saving you are looking for. Each one has its pros and cons, especially in terms of how long it take to get back the full functionalities versus the minimum mA used. As a reference, just the green power LED near the USB connector, consumes around 8 mA.

We begin with the typical sleep mode of the microcontroller and a manual pushbutton to wake it up.

This sketch demonstrates the usage of External Interrupts (on pins) to wakeup the SAMD21 of the MKR WiFi 1010 in sleep mode.

Sleep modes allow a significant drop in the power usage of a

Battery powered application can take advantage of these modes to enhance battery life significantly.

NOTE: if the processor is sleeping, a new sketch can't be uploaded. To overcome this, manually reset the board (usually with a single or double tap to the RESET button).

```

1  #include <ArduinoLowPower.h>
2  // Number of blinks per second
3  // Declare it volatile
4  volatile int repetitions = 0;
5  // Pin used to trigger the LED
6  const int pin = 8;
7  void setup() {
8      pinMode(LED_BUILTIN, OUTPUT);
9      // Attach a wakeup interrupt
10     LowPower.attachInterrupt(pin);
11 }
12 void loop() {
13     for (int i = 0; i < repetitions; i++) {
14         digitalWrite(LED_BUILTIN, HIGH);
15         delay(500);
16         digitalWrite(LED_BUILTIN, LOW);
17         delay(500);
18     }
19     // Triggers an infinite sleep
20     // The power consumption is very low
21     LowPower.sleep();
22 }
23 void repetitionsIncrease() {
24     // This function will increase the number of repetitions
25     // You can do some other things here
26     // Remember to avoid infinite loops
27     repetitions++;
28 }

```

In the next example we set the microcontroller to sleep for 8 seconds, when it wakes up, it does a double LED blink and goes back to sleep. This could be done using `delay()` to achieve the same result,

```
1  #include <ArduinoLowPower.h>
2
3  void setup() {
4      pinMode(LED_BUILTIN,
5          // Uncomment this function to enable the LED
6          // LowPower.attachInterrupt();
7      }
8
9  void loop() {
10     digitalWrite(LED_BUILTIN, HIGH);
11     delay(500);
12     digitalWrite(LED_BUILTIN, LOW);
13     delay(500);
14     digitalWrite(LED_BUILTIN, HIGH);
15     delay(500);
16     digitalWrite(LED_BUILTIN, LOW);
17     delay(500);
18     // Triggers a 2000 ms delay
19     // The power consumption is very low
20     LowPower.sleep(8000);
21 }
22
23 void dummy() {
24     // This function will be called by the
25     // You can do some low power tasks here
26     // Remember to avoid using the
27 }
```

WiFi.lowPowerMode() is capable of putting the Wi-Fi NINA module in a self managed low power mode. The data is transmitted and received, while the beacon is transmitted at regular intervals to keep the AP mode effective. To use this feature, it is enough to put this API in your sketch after the initialization procedure of the WiFi. Please note that there is no parameter passed to the function; to disable this low power mode you need to use the **WiFi.noLowPowerMode()** function.

In the following example we keep

seconds, allowing the NINA module to manage its power consumption autonomously.

```
1  #include <Wi-FiNINA.h>
2
3  #include "arduino_sec
4  //////////please enter yo
5  char ssid[] = SECRET_S
6  char pass[] = SECRET_P
7  int keyIndex = 0;
8
9  int status = WL_IDLE_S
10
11 WiFiClient client; //
12
13 char server[] = "exam
14
15 unsigned long lastConn
16 const unsigned long p
17
18 void setup() {
19     Serial.begin(9600);
20     while (!Serial) {
21         ; // wait for ser:
22     }
23
24     if (WiFi.status() ==
25         Serial.println("C
26         // don't continue
27         while (true);
28     }
```

The consumption of the NINA module alone can go down at around 30mA and this has to be added to the other components on your board.

A more radical way to reduce the consumption of the NINA module is to use **WiFi.end()** that turns off the radio part of the module. The microcontroller inside the module is still using some power, but this is the lowest level of consumption possible. This function in your

restart of the initialization process: `WiFi.begin()` or `WiFi.beginAP()` are required to restore the connections.

Conclusion

The four APIs covered in this tutorial allow you to minimize the power consumption of the MKR WiFi 1010 and get the most out of your rechargeable battery. The declared battery capacity, divided by the power used by the board will give you the expected number of hours your circuit can run on a single charge. To assess the real mA used we suggest that you use some of the nice and cheap USB power monitors available on the market.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

